ASSIGNMENT 6

Implementation of Data Integrity Constraints

Project: Airline Reservation System | Course: Database Management Systems

Objectives covered

- Implement primary keys and foreign keys for all relevant tables
- Apply UNIQUE, CHECK and DEFAULT constraints wherever required
- Explain the purpose of the constraints used
- Execute SQL statements that verify data integrity and relationships between tables are maintained

1. Introduction

Data integrity means that the data in the database is accurate, consistent and follows the business rules of the system. Instead of relying on every application screen to validate input, a relational database can enforce the rules itself through **integrity constraints**. In this assignment the constraints of the Airline Reservation System are listed and explained, and then **tested** by deliberately trying to break each rule. Every test was executed on the sample database of Assignment 7 (SQLite 3.45 with foreign keys switched on); the messages shown are the real database responses. The constraints were declared in the CREATE TABLE statements of Assignment 5 (*schema.sql*).

2. Types of Constraints Used and Their Purpose

Constraint	What it does	Why it is needed in an airline system
PRIMARY KEY	Uniquely identifies each row; cannot be NULL or duplicated	Every flight, passenger, booking and ticket must be individually addressable; foreign keys point to it
FOREIGN KEY	Value must exist in the parent table's key; defines the relationship and the action on delete/update	A ticket cannot exist for a non-existent flight or booking; prevents orphan records
UNIQUE	No two rows may share the value (NULLs allowed)	One passport / e-mail per passenger; one booking reference; one passenger per seat on a flight
NOT NULL	Value is mandatory	Fares, dates, names and routes must always be known
CHECK	Value must satisfy a logical condition	Fares are positive; a flight cannot land before it takes off; only valid statuses and classes
DEFAULT	Value used automatically when none is supplied	New bookings start as 'Pending'; new flights 'Scheduled'; nationality 'Indian'

3. Constraints Implemented, Table by Table

Table	Constraint (name)	Type	Purpose
airline	airline_id	PK	Identify an airline
airline	airline_name, iata_code	UNIQUE	No duplicate airlines or IATA codes
airport	airport_code	PK	3-letter IATA code identifies an airport
airport	chk_airport_code	CHECK	Code must be exactly 3 characters
aircraft	aircraft_id / registration_no	PK / UNIQUE	One row per physical aircraft; registration is unique
aircraft	chk_aircraft_seats	CHECK	1 ≤ total_seats ≤ 900
aircraft	chk_aircraft_status; status DEFAULT 'Active'	CHECK / DEFAULT	Only Active, Maintenance or Retired
aircraft	fk_aircraft_airline	FK → airline	Every aircraft belongs to an existing airline (RESTRICT)
flight	flight_id	PK	Identify a scheduled flight
flight	uq_flight_schedule	UNIQUE	(flight_number, departure_time): same flight number cannot depart twice at the same time
flight	chk_flight_route	CHECK	origin_code <> destination_code
flight	chk_flight_times	CHECK	arrival_time > departure_time
flight	chk_flight_fare	CHECK	base_fare > 0
flight	chk_flight_status; status DEFAULT 'Scheduled'	CHECK / DEFAULT	Scheduled, Delayed, Cancelled or Completed
flight	fk_flight_aircraft, fk_flight_origin, fk_flight_destination	FK	Aircraft and both airports must exist (RESTRICT / ON UPDATE CASCADE)
passenger	passenger_id	PK	Identify a passenger
passenger	email, passport_no	UNIQUE	One account / passport per person
passenger	chk_passenger_gender	CHECK	'M', 'F' or 'O'
passenger	nationality DEFAULT 'Indian'; registered_on DEFAULT CURRENT_DATE	DEFAULT	Sensible values when omitted
booking	booking_id; booking_ref	PK; UNIQUE	The 6-character reference given to the customer is unique
booking	chk_booking_status; status DEFAULT 'Pending'	CHECK / DEFAULT	Pending, Confirmed or Cancelled
booking	chk_booking_amount; total_amount DEFAULT 0	CHECK / DEFAULT	Total cannot be negative; starts at 0
booking	booking_date DEFAULT CURRENT_TIMESTAMP	DEFAULT	Time of booking recorded automatically
booking	fk_booking_passenger	FK → passenger	A booking is made by an existing passenger (RESTRICT)
ticket	ticket_id	PK	Identify a ticket
ticket	uq_ticket_seat	UNIQUE	(flight_id, seat_no): a seat can be sold only once per flight. Cancelled tickets set seat_no to NULL so the seat is released
ticket	chk_ticket_class; travel_class DEFAULT 'Economy'	CHECK / DEFAULT	Economy, Premium Economy or Business
ticket	chk_ticket_fare	CHECK	fare > 0
ticket	chk_ticket_status; status DEFAULT 'Booked'	CHECK / DEFAULT	Booked, Checked-In, Flown or Cancelled
ticket	chk_ticket_seat	CHECK	A seat number is mandatory unless the ticket is Cancelled
ticket	fk_ticket_booking	FK → booking	ON DELETE CASCADE: tickets die with their booking

Table	Constraint (name)	Type	Purpose
ticket	fk_ticket_flight, fk_ticket_passenger	FK	Flight and passenger must exist (RESTRICT)
payment	payment_id; transaction_ref	PK; UNIQUE	Each gateway transaction is recorded once
payment	chk_payment_amount	CHECK	amount > 0
payment	chk_payment_method; chk_payment_status; status DEFAULT 'Pending'	CHECK / DEFAULT	Valid payment methods and statuses
payment	fk_payment_booking	FK → booking	A payment always belongs to an existing booking (RESTRICT)

4. Testing the Constraints

Each test attempts an operation that **should** be refused (or, for the DEFAULT and cascade tests, should have a particular effect). A rejected statement leaves the data untouched. Tests that do change data run inside a transaction that is rolled back afterwards.

4.1 Primary key constraints

Test 1: Duplicate primary key (airline_id = 1 already exists)

SQL statement
```
INSERT INTO airline (airline_id, airline_name, iata_code, country)
VALUES (1, 'Test Air', 'TA', 'India');
```

RESULT
```
ERROR: UNIQUE constraint failed: airline.airline_id
-> statement REJECTED by the database; no data was changed
```

Why: A primary key value must identify exactly one row. The database refuses the second row with id 1.

Test 2: Duplicate airport code (BLR already exists)

SQL statement
```
INSERT INTO airport (airport_code, airport_name, city, country)
VALUES ('BLR', 'Another Airport', 'Bengaluru', 'India');
```

RESULT
```
ERROR: UNIQUE constraint failed: airport.airport_code
-> statement REJECTED by the database; no data was changed
```

Why: The IATA code is the key of the airport table; the same code cannot be stored twice.

Test 3: Missing mandatory value (no e-mail)

SQL statement
```
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Test', 'User', NULL, '+91-9000000001', '1990-01-01', 'M', 'X1234567');
```

RESULT
```
ERROR: NOT NULL constraint failed: passenger.email
-> statement REJECTED by the database; no data was changed
```

Why: NOT NULL columns must always be supplied; a passenger without an e-mail is refused.

4.2 UNIQUE constraints

Test 4: Duplicate e-mail address

SQL statement
```
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Test', 'User', 'aarav.sharma@example.com', '+91-9000000001', '1990-01-01', 'M', 'X1234567');
```

RESULT
```
ERROR: UNIQUE constraint failed: passenger.email
-> statement REJECTED by the database; no data was changed
```

Why: Two passengers cannot register with the same e-mail address.

Test 5: Duplicate passport number

```
SQL statement
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Test', 'User', 'test.user@example.com', '+91-9000000001', '1990-01-01', 'M',
        (SELECT passport_no FROM passenger WHERE passenger_id = 1));
```

```
RESULT
ERROR: UNIQUE constraint failed: passenger.passport_no
-> statement REJECTED by the database; no data was changed
```

Why: A passport number identifies one real person, so it must not be repeated.

Test 6: Duplicate booking reference

```
SQL statement
INSERT INTO booking (booking_id, booking_ref, passenger_id)
VALUES (21, (SELECT booking_ref FROM booking WHERE booking_id = 1), 1);
```

```
RESULT
ERROR: UNIQUE constraint failed: booking.booking_ref
-> statement REJECTED by the database; no data was changed
```

Why: The booking reference is what the customer quotes at the counter; it must identify only one booking.

Test 7: Same flight number departing at the same time twice

```
SQL statement
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, '6E-501', 2, 'BLR', 'DEL', '2026-11-02 06:00:00', '2026-11-02 08:45:00', 5200);
```

```
RESULT
ERROR: UNIQUE constraint failed: flight.flight_number, flight.departure_time
-> statement REJECTED by the database; no data was changed
```

Why: A flight number may be reused on other dates, but not twice for the same departure time.

Test 8: Selling an already occupied seat (flight 1, seat 8A)

```
SQL statement
INSERT INTO ticket (ticket_id, booking_id, flight_id, passenger_id, seat_no, fare)
VALUES (31, 3, 1, 4, '8A', 5200);
```

```
RESULT
ERROR: UNIQUE constraint failed: ticket.flight_id, ticket.seat_no
-> statement REJECTED by the database; no data was changed
```

Why: UNIQUE (flight_id, seat_no) makes double-booking of a seat impossible, even if two agents try at the same time.

4.3 CHECK constraints

Test 9: Flight with a zero base fare

```
SQL statement
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, 'XX-100', 1, 'BLR', 'DEL', '2026-12-01 06:00:00', '2026-12-01 08:45:00', 0);
```

```
RESULT
ERROR: CHECK constraint failed: chk_flight_fare
-> statement REJECTED by the database; no data was changed
```

Why: chk_flight_fare requires base_fare > 0.

Test 10: Flight whose origin and destination are the same

```
SQL statement
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, 'XX-101', 1, 'BLR', 'BLR', '2026-12-01 06:00:00', '2026-12-01 08:45:00', 3000);
```

```
RESULT
ERROR: CHECK constraint failed: chk_flight_route
-> statement REJECTED by the database; no data was changed
```

Why: chk_flight_route (origin_code <> destination_code): a flight must go somewhere.

Test 11: Flight arriving before it departs

```
SQL statement
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, 'XX-102', 1, 'BLR', 'DEL', '2026-12-01 10:00:00', '2026-12-01 08:00:00', 3000);
```

```
RESULT
ERROR: CHECK constraint failed: chk_flight_times
-> statement REJECTED by the database; no data was changed
```

Why: chk_flight_times (arrival_time > departure_time) catches impossible schedules.

Test 12: Aircraft with 0 seats

```
SQL statement
INSERT INTO aircraft (aircraft_id, registration_no, model, total_seats, airline_id)
VALUES (12, 'VT-ZZZ', 'Airbus A320', 0, 1);
```

```
RESULT
ERROR: CHECK constraint failed: chk_aircraft_seats
-> statement REJECTED by the database; no data was changed
```

Why: chk_aircraft_seats limits capacity to the range 1 to 900.

Test 13: Invalid gender code 'X'

```
SQL statement
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Test', 'User', 'test.user@example.com', '+91-9000000001', '1990-01-01', 'X', 'X1234567');
```

```
RESULT
ERROR: CHECK constraint failed: chk_passenger_gender
-> statement REJECTED by the database; no data was changed
```

Why: chk_passenger_gender allows only 'M', 'F' or 'O'.

Test 14: Invalid ticket status

```
SQL statement
UPDATE ticket SET status = 'Lost' WHERE ticket_id = 1;
```

```
RESULT
ERROR: CHECK constraint failed: chk_ticket_status
-> statement REJECTED by the database; no data was changed
```

Why: The CHECK constraint is also applied to UPDATE statements, not only to INSERT.

Test 15: Negative payment amount

SQL statement
```
INSERT INTO payment (payment_id, booking_id, amount, method, transaction_ref)
VALUES (24, 1, -500, 'UPI', 'TXN000000001');
```

RESULT
```
ERROR: CHECK constraint failed: chk_payment_amount
-> statement REJECTED by the database; no data was changed
```

Why: chk_payment_amount (amount > 0). Refunds are recorded as separate positive rows with status 'Refunded'.

Test 16: Unsupported payment method

SQL statement
```
INSERT INTO payment (payment_id, booking_id, amount, method, transaction_ref)
VALUES (24, 1, 500, 'Cash', 'TXN000000002');
```

RESULT
```
ERROR: CHECK constraint failed: chk_payment_method
-> statement REJECTED by the database; no data was changed
```

Why: Only the methods in chk_payment_method are accepted by the online system.

Test 17: Active ticket without a seat number

SQL statement
```
INSERT INTO ticket (ticket_id, booking_id, flight_id, passenger_id, seat_no, fare, status)
VALUES (31, 3, 1, 4, NULL, 5200, 'Booked');
```

RESULT
```
ERROR: CHECK constraint failed: chk_ticket_seat
-> statement REJECTED by the database; no data was changed
```

Why: chk_ticket_seat: only Cancelled tickets may have no seat. This rule lets cancelled tickets release their seat while still preventing seat-less live tickets.

Test 18: Airport code of wrong length

SQL statement
```
INSERT INTO airport (airport_code, airport_name, city, country)
VALUES ('XY', 'Test Airport', 'Test City', 'India');
```

RESULT
```
ERROR: CHECK constraint failed: chk_airport_code
-> statement REJECTED by the database; no data was changed
```

Why: chk_airport_code ensures an IATA code has exactly 3 characters.

4.4 FOREIGN KEY constraints (relationships)

Test 19: Ticket for a flight that does not exist

SQL statement
```
INSERT INTO ticket (ticket_id, booking_id, flight_id, passenger_id, seat_no, fare)
VALUES (31, 3, 999, 4, '9A', 5200);
```

RESULT
```
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: ticket.flight_id must match an existing flight_id, so tickets can never become orphans.

Test 20: Flight to an unknown airport

```
SQL statement
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, 'XX-103', 1, 'BLR', 'ZZZ', '2026-12-01 06:00:00', '2026-12-01 08:45:00', 3000);
```

```
RESULT
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: The destination code ZZZ is not in the airport table.

Test 21: Booking for a non-existent passenger

```
SQL statement
INSERT INTO booking (booking_id, booking_ref, passenger_id)
VALUES (21, 'ZZZ999', 999);
```

```
RESULT
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: Every booking must be made by a registered passenger.

Test 22: Payment for a non-existent booking

```
SQL statement
INSERT INTO payment (payment_id, booking_id, amount, method, transaction_ref)
VALUES (24, 999, 1000, 'UPI', 'TXN000000003');
```

```
RESULT
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: A payment without a booking would be unexplainable money.

Test 23: Deleting an airline that still owns aircraft

```
SQL statement
DELETE FROM airline WHERE airline_id = 1;
```

```
RESULT
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: ON DELETE RESTRICT protects history: IndiGo cannot be removed while its aircraft, flights and tickets exist.

Test 24: Deleting a passenger who holds tickets

```
SQL statement
DELETE FROM passenger WHERE passenger_id = 1;
```

```
RESULT
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: Passenger records must be kept while tickets and bookings refer to them (needed for audit and legal reasons).

Test 25: Deleting a booking that has payments

SQL statement
```
DELETE FROM booking WHERE booking_id = 1;
```

RESULT
```
ERROR: FOREIGN KEY constraint failed
-> statement REJECTED by the database; no data was changed
```

Why: The booking has a payment row, and payment → booking is RESTRICT, so financial records cannot be orphaned. (Cancel the booking by setting its status instead.)

4.5 Referential actions: CASCADE

The two tests below change data and then ROLLBACK; the counts show the effect of the cascade rules.

Test 26: ON DELETE CASCADE (booking → ticket)

SQL
```
BEGIN;
INSERT INTO booking (booking_id, booking_ref, passenger_id) VALUES (22, 'TMP001', 1);
INSERT INTO ticket (ticket_id, booking_id, flight_id, passenger_id, seat_no, fare)
VALUES (32, 22, 2, 1, '20A', 5100), (33, 22, 2, 1, '20B', 5100);
SELECT COUNT(*) AS tickets_of_booking_22 FROM ticket WHERE booking_id = 22;   -- before
DELETE FROM booking WHERE booking_id = 22;
SELECT COUNT(*) AS tickets_of_booking_22 FROM ticket WHERE booking_id = 22;   -- after
ROLLBACK;
```

RESULT
```
BEFORE
+-----------------------+
| tickets_of_booking_22 |
+-----------------------+
|                     2 |
+-----------------------+
AFTER DELETE FROM booking
+-----------------------+
| tickets_of_booking_22 |
+-----------------------+
|                     0 |
+-----------------------+
```

Why: A ticket has no meaning without its booking, so deleting the booking automatically removes its two tickets and no orphan rows remain.

Test 27: ON UPDATE CASCADE (airport → flight)

```sql
BEGIN;
SELECT COUNT(*) AS flights_from_BLR FROM flight WHERE origin_code = 'BLR';
UPDATE airport SET airport_code = 'BNG' WHERE airport_code = 'BLR';
SELECT origin_code, COUNT(*) AS flights FROM flight
WHERE origin_code IN ('BLR','BNG') GROUP BY origin_code;
ROLLBACK;
```

```
RESULT
BEFORE
+------------------+
| flights_from_BLR |
+------------------+
|                9 |
+------------------+
AFTER UPDATE airport
+-------------+---------+
| origin_code | flights |
+-------------+---------+
| BNG         |       9 |
+-------------+---------+
```

Why: When the key of a parent row is corrected, ON UPDATE CASCADE rewrites every referencing flight automatically, so no flight is left pointing at a code that no longer exists.

4.6 DEFAULT constraints

Test 28: Values supplied automatically when columns are omitted

```
SQL
BEGIN;
INSERT INTO passenger (passenger_id, first_name, last_name, email, phone, date_of_birth, gender, passport_no)
VALUES (26, 'Test', 'User', 'test.user@example.com', '+91-9000000001', '1990-01-01', 'M', 'X1234567');
INSERT INTO booking (booking_id, booking_ref, passenger_id) VALUES (22, 'TMP002', 26);
INSERT INTO aircraft (aircraft_id, registration_no, model, total_seats, airline_id)
VALUES (12, 'VT-TST', 'Airbus A320', 180, 1);
INSERT INTO flight (flight_id, flight_number, aircraft_id, origin_code, destination_code,
                    departure_time, arrival_time, base_fare)
VALUES (21, 'XX-200', 12, 'BLR', 'DEL', '2026-12-01 06:00:00', '2026-12-01 08:45:00', 5000);
INSERT INTO ticket (ticket_id, booking_id, flight_id, passenger_id, seat_no, fare)
VALUES (32, 22, 21, 26, '1A', 5000);
INSERT INTO payment (payment_id, booking_id, amount, method, transaction_ref)
VALUES (24, 22, 5000, 'UPI', 'TXN000000099');

SELECT nationality, registered_on = DATE('now') AS registered_today FROM passenger WHERE passenger_id = 26;
SELECT status, total_amount, booking_date IS NOT NULL AS date_auto_filled FROM booking WHERE booking_id = 22;
SELECT f.status AS flight_status, a.status AS aircraft_status, t.travel_class,
       t.status AS ticket_status, p.status AS payment_status
FROM flight f JOIN aircraft a ON a.aircraft_id = f.aircraft_id
JOIN ticket t ON t.flight_id = f.flight_id JOIN payment p ON p.booking_id = t.booking_id
WHERE f.flight_id = 21;
ROLLBACK;
```

```
RESULT
+-------------+------------------+
| nationality | registered_today |
+-------------+------------------+
| Indian      |                1 |
+-------------+------------------+
+---------+--------------+------------------+
| status  | total_amount | date_auto_filled |
+---------+--------------+------------------+
| Pending |            0 |                1 |
+---------+--------------+------------------+
+---------------+-----------------+--------------+---------------+----------------+
| flight_status | aircraft_status | travel_class | ticket_status | payment_status |
+---------------+-----------------+--------------+---------------+----------------+
| Scheduled     | Active          | Economy      | Booked        | Pending        |
+---------------+-----------------+--------------+---------------+----------------+
```

Why: None of the columns above was given in the INSERT statements, yet each has a sensible value: nationality 'Indian', registration date = today, booking 'Pending' with total 0 and an automatic timestamp, flight 'Scheduled', aircraft 'Active', ticket 'Economy' / 'Booked', payment 'Pending'. (In MySQL, use CURDATE() instead of DATE('now').)

5. Summary of Constraint Tests

Test	Constraint	Scenario	Outcome	Result
1	PRIMARY KEY	Duplicate primary key (airline_id = 1 already exists)	Rejected	PASS
2	PRIMARY KEY	Duplicate airport code (BLR already exists)	Rejected	PASS
3	NOT NULL	Missing mandatory value (no e-mail)	Rejected	PASS
4	UNIQUE	Duplicate e-mail address	Rejected	PASS
5	UNIQUE	Duplicate passport number	Rejected	PASS
6	UNIQUE	Duplicate booking reference	Rejected	PASS
7	UNIQUE (composite)	Same flight number departing at the same time twice	Rejected	PASS
8	UNIQUE (composite)	Selling an already occupied seat (flight 1, seat 8A)	Rejected	PASS
9	CHECK	Flight with a zero base fare	Rejected	PASS
10	CHECK	Flight whose origin and destination are the same	Rejected	PASS
11	CHECK	Flight arriving before it departs	Rejected	PASS
12	CHECK	Aircraft with 0 seats	Rejected	PASS
13	CHECK	Invalid gender code 'X'	Rejected	PASS
14	CHECK	Invalid ticket status	Rejected	PASS
15	CHECK	Negative payment amount	Rejected	PASS
16	CHECK	Unsupported payment method	Rejected	PASS
17	CHECK	Active ticket without a seat number	Rejected	PASS
18	CHECK	Airport code of wrong length	Rejected	PASS
19	FOREIGN KEY	Ticket for a flight that does not exist	Rejected	PASS
20	FOREIGN KEY	Flight to an unknown airport	Rejected	PASS
21	FOREIGN KEY	Booking for a non-existent passenger	Rejected	PASS
22	FOREIGN KEY	Payment for a non-existent booking	Rejected	PASS
23	FOREIGN KEY (RESTRICT)	Deleting an airline that still owns aircraft	Rejected	PASS
24	FOREIGN KEY (RESTRICT)	Deleting a passenger who holds tickets	Rejected	PASS
25	FOREIGN KEY (RESTRICT)	Deleting a booking that has payments	Rejected	PASS
26	FOREIGN KEY (CASCADE)	Deleting a booking removes its tickets	Cascaded	PASS
27	FOREIGN KEY (CASCADE)	Changing an airport code updates its flights	Cascaded	PASS
28	DEFAULT	Omitted columns receive default values	Defaults applied	PASS

28 of 28 tests behaved as required: invalid operations were rejected and the cascade and default rules worked.

6. Verifying Data Integrity and Relationships in the Existing Data

The following verification queries search the loaded sample data for violations of the business rules. For each query an **empty result** (0 rows) means the rule holds.

V1: Tickets whose flight does not exist (orphans)

```sql
SELECT t.ticket_id FROM ticket t LEFT JOIN flight f ON t.flight_id = f.flight_id WHERE f.flight_id IS NULL
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: LEFT JOIN keeps every ticket; a NULL on the flight side would reveal a missing parent. Result: no violations.

V2: Tickets whose booking does not exist

```sql
SELECT t.ticket_id FROM ticket t LEFT JOIN booking b ON t.booking_id = b.booking_id WHERE b.booking_id IS NULL
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Same anti-join pattern for the booking relationship. Result: no violations.

V3: Seats sold more than once on the same flight

```sql
SELECT flight_id, seat_no, COUNT(*) AS times_sold FROM ticket WHERE seat_no IS NOT NULL
GROUP BY flight_id, seat_no HAVING COUNT(*) > 1
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Re-checks the UNIQUE (flight_id, seat_no) rule using GROUP BY ... HAVING. Result: no violations.

V4: Booking totals that differ from the sum of their ticket fares

```sql
SELECT b.booking_id, b.total_amount, SUM(t.fare) AS tickets_sum
FROM booking b JOIN ticket t ON t.booking_id = b.booking_id
GROUP BY b.booking_id, b.total_amount HAVING b.total_amount <> SUM(t.fare)
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: total_amount is controlled redundancy (Assignment 3); this query proves it is still consistent. Result: no violations.

V5: Confirmed bookings not fully paid

```sql
SELECT b.booking_id, b.total_amount, COALESCE(SUM(p.amount), 0) AS paid
FROM booking b LEFT JOIN payment p ON p.booking_id = b.booking_id AND p.status = 'Success'
WHERE b.status = 'Confirmed'
GROUP BY b.booking_id, b.total_amount HAVING COALESCE(SUM(p.amount), 0) <> b.total_amount
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Every Confirmed booking must have successful payments equal to its total. Result: no violations.

V6: Cancelled bookings that still have active tickets

```sql
SELECT t.ticket_id FROM ticket t JOIN booking b ON b.booking_id = t.booking_id
WHERE b.status = 'Cancelled' AND t.status <> 'Cancelled'
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Cancelling a booking must cancel all its tickets. Result: no violations.

V7: Flights with more active tickets than aircraft seats

```sql
SELECT f.flight_id, a.total_seats, COUNT(*) AS sold
FROM ticket t JOIN flight f ON f.flight_id = t.flight_id JOIN aircraft a ON a.aircraft_id = f.aircraft_id
WHERE t.status <> 'Cancelled' GROUP BY f.flight_id, a.total_seats HAVING COUNT(*) > a.total_seats
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Capacity rule that a simple CHECK cannot express because it spans three tables; verified by query (a trigger can enforce it). Result: no violations.

V8: Cancelled tickets that still hold a seat

```sql
SELECT ticket_id, seat_no FROM ticket WHERE status = 'Cancelled' AND seat_no IS NOT NULL
```

```
OUTPUT
Empty set (0 rows)
```

Explanation: Ensures released seats are really free. Result: no violations.

Positive check: relationships return the expected number of rows

```sql
SELECT (SELECT COUNT(*) FROM ticket) AS tickets,
       (SELECT COUNT(*) FROM ticket t JOIN booking b   ON t.booking_id   = b.booking_id
                                      JOIN flight f    ON t.flight_id    = f.flight_id
                                      JOIN passenger p ON t.passenger_id = p.passenger_id) AS tickets_joined_to_parents;
```

```
OUTPUT
+---------+--------------------------+
| tickets | tickets_joined_to_parents |
+---------+--------------------------+
|      30 |                       30 |
+---------+--------------------------+
1 row in set
```

Explanation: Both counts are equal (30), so every ticket is linked to a booking, a flight and a passenger. No row is lost when the tables are joined.

```sql
Built-in referential integrity check
PRAGMA foreign_key_check;
```

```
OUTPUT
foreign_key_check -> 0 violation(s)
```

7. Conclusion

The Airline Reservation System enforces its main business rules inside the database: primary keys and foreign keys keep every record identifiable and connected; UNIQUE constraints prevent duplicate passports, e-mails, booking references and seat sales; CHECK constraints reject impossible fares, routes, times and statuses; and DEFAULT values give new rows sensible starting values. All twenty-eight tests produced the intended result, and the verification queries confirm that the existing data contains no orphan rows, duplicate seats or inconsistent totals. Rules that span several tables, such as aircraft capacity, are verified by query and could be enforced with triggers in a production system.